

TITLE: IMPLEMENTATION OF BRESENHAM'S LINE DRAWING ALGORITHM

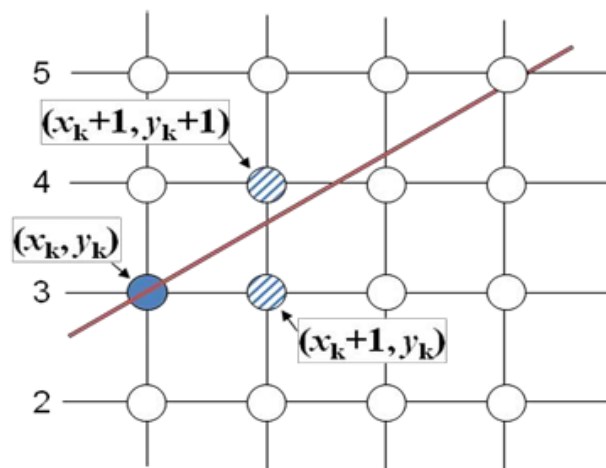
INTRODUCTION

Bresenham's line drawing algorithm is a widely used computer graphics method for drawing a straight line between two given points on a raster screen. It provides a highly efficient way to determine which pixels should be activated to form a line using only integer calculations (no floating-point values). At each step along the x-axis, the algorithm decides between two possible y-values using a decision parameter, choosing the pixel closest to the true line path.

This technique is faster than DDA because it uses only additions and comparisons, making it ideal for real-time rendering in computer graphics.

Basic concept:

- Move across the x axis in unit intervals and at each step choose between two different y coordinates



- For example, from position (2, 3) we have to choose between (3, 3) and (3, 4). We would like the point that is closer to the original line
- So we have to take decision to choose next point. So next pixels are selected based on the value of decision parameter p. The equations are given in below algorithm.

ALGORITHM

Step 1: Start the algorithm.

Step 2: Input the two end-points of the line (x_0, y_0) and (x_1, y_1) .

Store the left starting point in (x_0, y_0) .

Step 3: Plot the first point (x0, y0) using putpixel().

Step 4: Calculate:

$$\Delta x = x1 - x0$$

$$\Delta y = y1 - y0$$

Compute constants: $2\Delta y$ and $(2\Delta y - 2\Delta x)$

Step 5: Compute the initial decision parameter:

$$p0 = 2\Delta y - \Delta x$$

Step 6: Set $k = 0$ and $x = x0, y = y0$

Step 7: Repeat while $k < \Delta x$

a. If $p_k < 0$

Next point = $(x + 1, y)$

$$p_{k+1} = p_k + 2\Delta y$$

b. Else

Next point = $(x + 1, y + 1)$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

c. Plot the selected next point

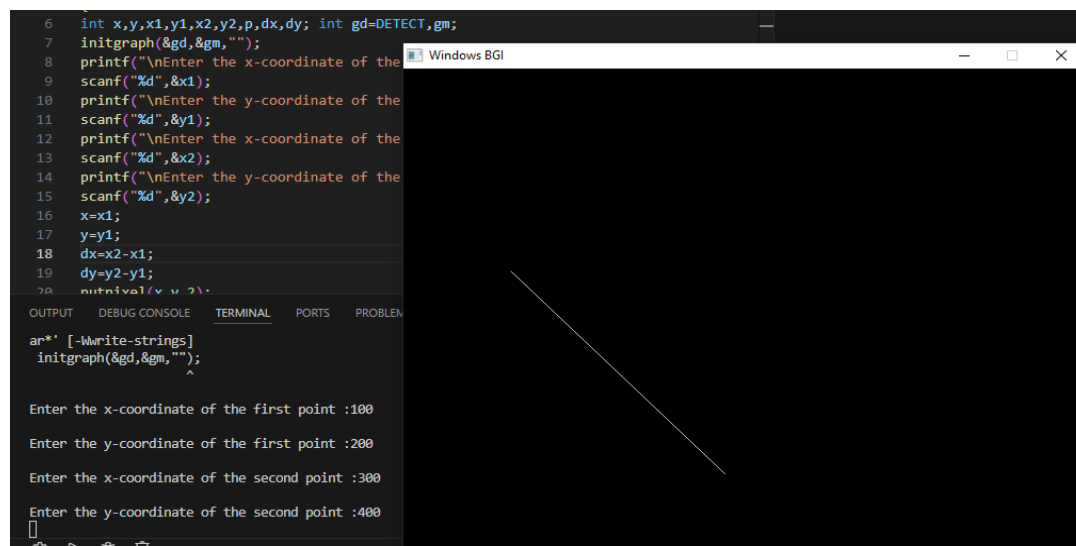
d. Increment x and/or y accordingly

e. Increment k

Step 8: Stop the algorithm.

Note: The above steps apply when slope $|m| < 1$. For slopes greater than 1, the algorithm is modified by stepping along the y-axis instead of the x-axis.

OUTPUT



```
6  int x,y,x1,y1,x2,y2,p,dx,dy; int gd=DETECT, gm;
7  initgraph(&gd,&gm,"");
8  printf("\nEnter the x-coordinate of the first point :");
9  scanf("%d",&x1);
10 printf("\nEnter the y-coordinate of the first point :");
11 scanf("%d",&y1);
12 printf("\nEnter the x-coordinate of the second point :");
13 scanf("%d",&x2);
14 printf("\nEnter the y-coordinate of the second point :");
15 scanf("%d",&y2);
16 x=x1;
17 y=y1;
18 dx=x2-x1;
19 dy=y2-y1;
20 p=2*dy-dx;

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS
ar** [-Wwrite-strings]
initgraph(&gd,&gm,"");
^
Enter the x-coordinate of the first point :100
Enter the y-coordinate of the first point :200
Enter the x-coordinate of the second point :300
Enter the y-coordinate of the second point :400
[]
```

CONCLUSION

Bresenham's line drawing algorithm provides an accurate and efficient method to draw straight lines using only integer arithmetic. By incrementally selecting the nearest pixel through a decision-making parameter, it avoids complex calculations and improves drawing speed. It is a key foundational algorithm in computer graphics and is widely used for rendering lines in graphical applications.